

Venus - VBep20 Donation Patch

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Token	Documentation quality	High	
Timeline	2026-03-18 through 2026-03-20	Test quality	High	
Language	Solidity	Total Findings	2	Acknowledged: 2
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Specification	None	Medium severity findings ⓘ	0	
Source Code	VenusProtocol/venus-protocol ⓘ #0a905ab ⓘ	Low severity findings ⓘ	1	Acknowledged: 1
Auditors	- Rabib Islam Senior Auditing Engineer - Paul Clemson Auditing Engineer	Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	1	Acknowledged: 1

Summary of Findings

This audit focused on a targeted remediation to the VBep20 cash-accounting model. Under the prior design, economically relevant market state depended on the contract's raw underlying-token balance, which meant direct token transfers to a market could affect downstream values such as exchange-rate and liquidity-related calculations without passing through the protocol's normal accounting controls. The purpose of this patch is to remove that dependency by introducing an internally tracked cash value and ensuring that only protocol-mediated transfers can change it.

The patch materially improves the security of the VBep20 cash-accounting model by replacing economically relevant reads of raw `balanceOf(address(this))` with an internally tracked `internalCash` value. In steady state, this removes the direct donation-based accounting amplifier targeted by the remediation. Incoming and outgoing token flows now update tracked cash using actual transfer deltas, and the accompanying tests provide strong coverage of mint, redeem, borrow, repay, reserve, donation, and flash-loan behavior under the new model.

Our remaining concerns are regarding migration and privileged operator surfaces rather than in the post-migration accounting design. In particular, `sweepTokenAndSync()` should be executed in the same transaction as each VBep20 implementation upgrade, and the function's `transferAmount` parameter should be handled carefully because it can sweep tracked market cash if misused. It should also be understood that syncing `internalCash` to the live underlying balance at upgrade time will preserve any donations already embedded in a market's balance. Subject to an atomic upgrade-and-sync rollout and appropriate operational care around the new sweep function, we consider the patch adequate to address the targeted donation-attack vulnerability.

Fix-Review Update 2026-03-20:

The client elected to acknowledge both of the issues put forward in this report.

ID	DESCRIPTION	SEVERITY	STATUS
VEN-1	<code>sweepTokenAndSync()</code> Can Sweep Tracked Market Cash because <code>transferAmount</code> Is Unconstrained	Low ⓘ	Acknowledged
VEN-2	Upgrade Safety Depends on Operator Sequencing Rather than Self-Initializing Code	Informational ⓘ	Acknowledged

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.



Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

The scope includes contracts changed in the branch feat/VPD-808.

Files Included

- `contracts/Tokens/VTokens/VBep20.sol`
- `contracts/Tokens/VTokens/VTokenInterfaces.sol`

Operational Considerations

The implementation being audited will require the application of upgrades. Upgrades are capable of arbitrarily changing smart contract logic. Done improperly, they may result in the loss of user funds.

Moreover, it should be understood that the application of this upgrade does not ensure that there are not any donated tokens inside the `VBep20` contracts at the time of upgrade; hence, it is possible that the `internalCash` variable will come to include both donated tokens and non-donated tokens.

Key Actors And Their Capabilities

The `admin` address will become capable of calling `sweepTokenAndSync()`, which is capable of transferring an arbitrary amount of the underlying token from `VBep20` to the admin.

Findings

VEN-1

`sweepTokenAndSync()` Can Sweep Tracked Market Cash because `transferAmount` Is Unconstrained

• Low ⓘ Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

We acknowledge the observation. The `sweepTokenAndSync` function is admin-only and the admin is a governance timelock, so any call requires a governance vote. The `transferAmount` parameter is intentionally flexible to allow recovery of arbitrary donation amounts without requiring multiple transactions. The risk of accidentally sweeping tracked user cash is mitigated by the governance process — every parameter is publicly visible and reviewed before execution. We accept this as a known trade-off in favor of operational flexibility.

File(s) affected: `VBep20.sol`

Description: `sweepTokenAndSync(uint256 transferAmount)` is a migration/sweep helper for syncing `internalCash` and recovering excess donations, but it does not enforce that `transferAmount` is limited to true excess underlying. Instead, it transfers any requested amount of underlying to `admin` and then redefines `internalCash` to the new on-chain balance. That means the parameter is broader than necessary for the stated purpose: a benign `admin` can accidentally sweep tracked user cash rather than only inert donations, and the contract will accept that as the new canonical cash balance. This is not a privileged-theft finding under the audit rules, but it is an avoidable footgun and a least-privilege design miss.

Recommendation: Consider implementing one of two following remediations:

1. Add a check which requires `transferAmount <= IERC20(underlying).balanceOf(address(this)) - internalCash` before transferring;
2. Remove the `transferAmount` parameter entirely and compute excess internally as `IERC20(underlying).balanceOf(address(this)) - internalCash`, and transfer this amount to the admin.

VEN-2

Upgrade Safety Depends on Operator Sequencing Rather than Self-Initializing Code

• Informational ⓘ Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

we make sure `vToken` upgrade and `internalCash` value is updated in a single transaction same VIP
[<https://github.com/VenusProtocol/vips/blob/9b434ba29f1d875da0130d042485a706fe34007f/vips/vip-608/bscmainnet.ts>]

File(s) affected: `VBep20Delegate.sol`, `VBep20Delegator.sol`, `VBep20.sol`

Description: The new `internalCash` invariant is not established by the upgrade itself. `_setImplementation()` delegate-calls `_becomeImplementation(bytes)`, but `_becomeImplementation(bytes)` ignores its `data` parameter and does not offer a means to initialize `internalCash`. The storage comment explicitly says `internalCash` “must be initialized via `sweepTokenAndSync()` after upgrade.” This means a freshly upgraded live `VBep20Delegator` is temporarily in an invalid state until `sweepTokenAndSync(0)` runs, even though `vToken` immediately consumes `getCashPrior()` in exchange-rate and interest paths. With the client’s stated same-transaction upgrade procedure this should not become exploitable in practice, but from a secure-upgrade-design standpoint the fix relies on operational discipline rather than code-enforced initialization.

Recommendation: Initialize `internalCash` inside `_becomeImplementation(bytes)` or decode `becomeImplementationData` and set it there. The upgrade should leave the market in a valid state without requiring a separate follow-up call.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.

- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Test Suite Results

The test data was obtained by running `npx hardhat coverage`.

This patch introduces targeted tests for the new `internalCash` accounting model and for the migration procedure on live markets.

- `tests/hardhat/Fork/donationAttack.ts` uses a BSC mainnet fork to demonstrate that direct token transfers inflate exchange rates under the old balance-based model, then verifies that after upgrading each listed `VBep20` market and calling `sweepTokenAndSync(0)`, direct donations no longer change `exchangeRateStored()`, `getCash()`, or `internalCash`. The same suite also checks that `internalCash` is initialized to the live balance and that normal mint and redeem operations continue to move tracked cash in the expected direction.
- `tests/hardhat/Fork/vTokenStorageChecks.ts` upgrades a broad set of mainnet `VBep20Delegator` markets on fork and verifies that the proxy storage layout remains intact after upgrade, that `internalCash` equals the actual underlying balance after synchronization, and that mint, redeem, borrow, repay, donation, and `accrueInterest()` flows preserve the key invariant `internalCash == underlying.balanceOf(vToken)` after each operation.
- `tests/hardhat/VToken/sweepTokenAndSyncCash.ts` adds unit coverage for the new admin sync/sweep function itself, including admin-only access control, zero-balance sync, idempotent re-sync behavior, event emission, exact-excess sweeping after one or more donations, and reversion when the requested sweep amount exceeds the contract's token balance.
- Existing suites were also updated to exercise adjacent paths affected by the patch. In particular, `tests/hardhat/Comptroller/Diamond/flashLoan.ts` covers full-repayment and partial-repayment flash-loan scenarios, protocol-fee accounting, and debt-position fallback under the new tracked-cash model, while `tests/hardhat/VToken/reservesTest.ts` verifies reserve-reduction behavior using `internalCash` rather than raw token balance.

Version

=====

> solidity-coverage: v0.8.14

Instrumenting for coverage...

=====

> Admin/VBNBAdmin.sol

> Admin/VBNBAdminStorage.sol

> Comptroller/ComptrollerInterface.sol

> Comptroller/ComptrollerLensInterface.sol

> Comptroller/ComptrollerStorage.sol

> Comptroller/Diamond/Diamond.sol

> Comptroller/Diamond/DiamondConsolidated.sol

> Comptroller/Diamond/facets/FacetBase.sol

> Comptroller/Diamond/facets/FlashLoanFacet.sol

> Comptroller/Diamond/facets/MarketFacet.sol

> Comptroller/Diamond/facets/PolicyFacet.sol

> Comptroller/Diamond/facets/RewardFacet.sol

> Comptroller/Diamond/facets/SetterFacet.sol

> Comptroller/Diamond/facets/XVSRewardsHelper.sol

> Comptroller/Diamond/interfaces/IDiamondCut.sol

> Comptroller/Diamond/interfaces/IFacetBase.sol

> Comptroller/Diamond/interfaces/IFlashLoanFacet.sol

> Comptroller/Diamond/interfaces/IMarketFacet.sol

> Comptroller/Diamond/interfaces/IPolicyFacet.sol

> Comptroller/Diamond/interfaces/IRewardFacet.sol

> Comptroller/Diamond/interfaces/ISetterFacet.sol

> Comptroller/legacy/ComptrollerInterfaceR1.sol

> Comptroller/legacy/ComptrollerLensInterfaceR1.sol

> Comptroller/legacy/ComptrollerStorageR1.sol

- > Comptroller/legacy/Diamond/DiamondConsolidatedR1.sol
- > Comptroller/legacy/Diamond/DiamondR1.sol
- > Comptroller/legacy/Diamond/facets/FacetBaseR1.sol
- > Comptroller/legacy/Diamond/facets/MarketFacetR1.sol
- > Comptroller/legacy/Diamond/facets/PolicyFacetR1.sol
- > Comptroller/legacy/Diamond/facets/RewardFacetR1.sol
- > Comptroller/legacy/Diamond/facets/SetterFacetR1.sol
- > Comptroller/legacy/Diamond/facets/XVSRewardsHelperR1.sol
- > Comptroller/legacy/Diamond/interfaces/IDiamondCutR1.sol
- > Comptroller/legacy/Diamond/interfaces/IMarketFacetR1.sol
- > Comptroller/legacy/Diamond/interfaces/IPolicyFacetR1.sol
- > Comptroller/legacy/Diamond/interfaces/IRewardFacetR1.sol
- > Comptroller/legacy/Diamond/interfaces/ISetterFacetR1.sol
- > Comptroller/Types/PoolMarketId.sol
- > Comptroller/Unitroller.sol
- > DelegateBorrowers/MoveDebtDelegate.sol
- > DelegateBorrowers/SwapDebtDelegate.sol
- > external/IProtocolShareReserve.sol
- > external/IWBNB.sol
- > FlashLoan/interfaces/IFlashLoanReceiver.sol
- > Governance/TokenRedeemer.sol
- > Governance/VTreasury.sol
- > Governance/VTreasuryV8.sol
- > InterestRateModels/InterestRateModel.sol
- > InterestRateModels/InterestRateModelV8.sol
- > InterestRateModels/JumpRateModel.sol
- > InterestRateModels/TwoKinksInterestRateModel.sol
- > InterestRateModels/WhitePaperInterestRateModel.sol
- > InterfacesV8.sol
- > Lens/ComptrollerLens.sol
- > Lens/SnapshotLens.sol
- > Lens/VenusLens.sol
- > lib/approveOrRevert.sol
- > lib/Currency.sol
- > Liquidator/BUSDLiquidator.sol
- > Liquidator/Liquidator.sol
- > Liquidator/LiquidatorStorage.sol
- > Oracle/PriceOracle.sol
- > PegStability/IVAI.sol
- > PegStability/PegStability.sol
- > Swap/interfaces/CustomErrors.sol
- > Swap/interfaces/InterfaceComptroller.sol
- > Swap/interfaces/IPancakePair.sol
- > Swap/interfaces/IPancakeSwapV2Factory.sol
- > Swap/interfaces/IPancakeSwapV2Router.sol
- > Swap/interfaces/IVBNB.sol
- > Swap/interfaces/IVtoken.sol
- > Swap/interfaces/IWBNB.sol
- > Swap/IRouterHelper.sol
- > Swap/lib/PancakeLibrary.sol
- > Swap/lib/TransferHelper.sol
- > Swap/RouterHelper.sol
- > Swap/SwapRouter.sol
- > Tokens/EIP20Interface.sol
- > Tokens/EIP20NonStandardInterface.sol
- > Tokens/Prime/Interfaces/InterfaceComptroller.sol
- > Tokens/Prime/Interfaces/IPoolRegistry.sol
- > Tokens/Prime/Interfaces/IPrime.sol
- > Tokens/Prime/Interfaces/IPrimeLiquidityProvider.sol
- > Tokens/Prime/Interfaces/IVToken.sol
- > Tokens/Prime/Interfaces/IXSVault.sol
- > Tokens/Prime/IPrime.sol
- > Tokens/Prime/IPrimeV5.sol
- > Tokens/Prime/libs/FixedMath.sol
- > Tokens/Prime/libs/FixedMath0x.sol
- > Tokens/Prime/libs/Scores.sol
- > Tokens/Prime/Prime.sol
- > Tokens/Prime/PrimeLiquidityProvider.sol
- > Tokens/Prime/PrimeLiquidityProviderStorage.sol
- > Tokens/Prime/PrimeStorage.sol
- > Tokens/test/IERC20NonStandard.sol
- > Tokens/VAI/IVAI.sol


```
> Tokens/VAI/lib.sol
> Tokens/VAI/VAI.sol
> Tokens/VAI/VAIController.sol
> Tokens/VAI/VAIControllerInterface.sol
> Tokens/VAI/VAIControllerStorage.sol
> Tokens/VAI/VAIUnitroller.sol
> Tokens/VRT/VRT.sol
> Tokens/VRT/VRTConverter.sol
> Tokens/VRT/VRTConverterProxy.sol
> Tokens/VRT/VRTConverterStorage.sol
> Tokens/VTokens/legacy/ComptrollerInterface.sol
> Tokens/VTokens/legacy/IProtocolShareReserveV5.sol
> Tokens/VTokens/legacy/Uutils/CarefulMath.sol
> Tokens/VTokens/legacy/Uutils/ErrorReporter.sol
> Tokens/VTokens/legacy/Uutils/Exponential.sol
> Tokens/VTokens/legacy/Uutils/ExponentialNoError.sol
> Tokens/VTokens/legacy/VBep20DelegateR1.sol
> Tokens/VTokens/legacy/VBep20DelegatorR1.sol
> Tokens/VTokens/legacy/VBep20R1.sol
> Tokens/VTokens/legacy/VTokenInterfaceR1.sol
> Tokens/VTokens/legacy/VTokenR1.sol
> Tokens/VTokens/legacy/VTokenStorageR1.sol
> Tokens/VTokens/VBep20.sol
> Tokens/VTokens/VBep20Delegate.sol
> Tokens/VTokens/VBep20Delegator.sol
> Tokens/VTokens/VBep20Immutable.sol
> Tokens/VTokens/VBNB.sol
> Tokens/VTokens/VToken.sol
> Tokens/VTokens/VTokenInterfaces.sol
> Tokens/XVS/IXVS.sol
> Tokens/XVS/IXVSVesting.sol
> Tokens/XVS/XVS.sol
> Tokens/XVS/XVSVesting.sol
> Tokens/XVS/XVSVestingProxy.sol
> Tokens/XVS/XVSVestingStorage.sol
> Uutils/Address.sol
> Uutils/CarefulMath.sol
> Uutils/CheckpointView.sol
> Uutils/Context.sol
> Uutils/ECDSA.sol
> Uutils/ErrorReporter.sol
> Uutils/Exponential.sol
> Uutils/ExponentialNoError.sol
> Uutils/IBEP20.sol
> Uutils/Ownable.sol
> Uutils/Owned.sol
> Uutils/ReentrancyGuardTransient.sol
> Uutils/SafeBEP20.sol
> Uutils/SafeCast.sol
> Uutils/SafeMath.sol
> Uutils/Tokenlock.sol
> Uutils/TransientSlot.sol
> VAIVault/VAIVault.sol
> VAIVault/VAIVaultErrorReporter.sol
> VAIVault/VAIVaultProxy.sol
> VAIVault/VAIVaultStorage.sol
> VRTVault/VRTVault.sol
> VRTVault/VRTVaultProxy.sol
> VRTVault/VRTVaultStorage.sol
> XSVVault/XVSStore.sol
> XSVVault/XSVVault.sol
> XSVVault/XSVVaultErrorReporter.sol
> XSVVault/XSVVaultProxy.sol
> XSVVault/XSVVaultStorage.sol
```

Coverage skipped for:

=====

```
> test/AccessControlManagerMock.sol
> test/assets/BEP20BitcoinCash.sol
> test/assets/BEP20BTC.sol
> test/assets/BEP20Ethereum.sol
```

```
> test/assets/BEP20XRP.sol
> test/BadFlashLoanReceiver.sol
> test/BEP20.sol
> test/BorrowDebtFlashLoanReceiver.sol
> test/ComptrollerHarness.sol
> test/ComptrollerMock.sol
> test/ComptrollerMockR1.sol
> test/ComptrollerScenario.sol
> test/Const.sol
> test/Counter.sol
> test/DiamondHarness.sol
> test/DiamondHarnessInterface.sol
> test/EvilToken.sol
> test/EvilXDelegator.sol
> test/EvilXToken.sol
> test/FalseMarker.sol
> test/Fauceteer.sol
> test/FaucetToken.sol
> test/FeeToken.sol
> test/FixedPriceOracle.sol
> test/FlashLoanReceiverBase.sol
> test/InsufficientRepaymentFlashLoanReceiver.sol
> test/InterestRateModelHarness.sol
> test/LiquidatorHarness.sol
> test/MathHelpers.sol
> test/MockDeflationaryToken.sol
> test/MockFlashLoanReceiver.sol
> test/MockMCD.sol
> test/MockProtocolShareReserve.sol
> test/MockToken.sol
> test/MockVBNB.sol
> test/PrimeScenario.sol
> test/SimplePriceOracle.sol
> test/Structs.sol
> test/VAIControllerHarness.sol
> test/VAIHarness.sol
> test/VBep20Harness.sol
> test/VBep20MockDelegate.sol
> test/VRTConverterHarness.sol
> test/VRTVaultHarness.sol
> test/WBNB.sol
> test/WBTC.sol
> test/XVSHarness.sol
> test/XVSVaultHarness.sol
> test/XVSVaultScenario.sol
> test/XVSVestingHarness.sol
```

Compilation:

=====

Generating typings for: 397 artifacts in dir: typechain for target: ethers-v5

Successfully generated 980 typings!

Compiled 268 Solidity files successfully (evm targets: cancun, istanbul).

Network Info

=====

> HardhatEVM: v2.22.18

> network: hardhat

VBNBAdmin

✓ set VBNBAdmin as vBNB admin

harvest income

✓ reduce BNB reserves

set interest rate model

✓ setInterestRateModel

Comptroller

_initializeMarket

✓ Supply and borrow state after initializing the market in the pool

_setVenusSpeeds

- ✓ Revert on invalid supplySpeeds input
- ✓ Revert on invalid borrowSpeeds input
- ✓ Revert for unlisted market
- ✓ Revert on invalid borrowSpeeds input
- ✓ Updating non-zero speeds after setting it zero

Comptroller

_setAccessControlManager

- ✓ Reverts **if** called by non-admin
- ✓ Reverts **if** ACM is zero address
- ✓ Sets ACM address in storage
- ✓ should revert on same value

Access Control

setCollateralFactor

- ✓ Should have AccessControl

setLiquidationIncentive

- ✓ Should have AccessControl

setMarketBorrowCaps

- ✓ Should have AccessControl

setMarketSupplyCaps

- ✓ Should have AccessControl

setProtocolPaused

- ✓ Should have AccessControl

setActionsPaused

- ✓ Should have AccessControl

_supportMarket

- ✓ Should have AccessControl

supportMarket

- ✓ Should have AccessControl

seizeVenus

- ✓ Should have AccessControl

Comptroller: assetListTest

enterMarkets

- ✓ properly emits events (41ms)
- ✓ adds to the asset list only once (79ms)
- ✓ the market must be listed for **add** to succeed (45ms)
- ✓ returns a list of codes mapping to user's ultimate membership in given addresses (43ms)

exitMarket

- ✓ doesn't let you exit **if** you have a borrow balance (63ms)
- ✓ rejects unless redeem allowed (123ms)
- ✓ accepts when you're **not** in the market already (66ms)
- ✓ properly removes when there's only one asset (119ms)
- ✓ properly removes when there's only two assets, removing the first (144ms)
- ✓ properly removes when there's only two assets, removing the second (135ms)
- ✓ properly removes when there's only three assets, removing the first (155ms)
- ✓ properly removes when there's only three assets, removing the second (151ms)
- ✓ properly removes when there's only three assets, removing the third (147ms)

entering from borrowAllowed

- ✓ enters when called by a vtoken (51ms)
- ✓ reverts when called by **not** a vtoken
- ✓ adds to the asset list only once (66ms)

unlistMarkets

- ✓ properly emits events **and** unlist market (78ms)
- ✓ reverts when unlisting **not** a listed market (67ms)

Comptroller

constructor

- ✓ on success it sets admin to creator **and** pendingAdmin is unset (1155ms)

setLiquidationIncentive

- ✓ fails **if** incentive is less than 1e18
- ✓ accepts a valid incentive **and** emits a NewLiquidationIncentive event
- ✓ should revert on same values

_setVenusVAIVaultRate

- ✓ should revert on same values

_setVAIVaultInfo

- ✓ should revert on same values

_setVAIController

- ✓ should revert on same values

_setVAIMintRate

- ✓ should revert on same values

_setLiquidatorContract

- ✓ should revert on same values
- ✓ should revert on zero address

_setPauseGuardian

- ✓ should revert on same values

_setVenusSpeeds

- ✓ ensure non zero address for venus speeds

_setPriceOracle

- ✓ fails **if** called by non-admin
- ✓ accepts a valid price oracle **and** emits a NewPriceOracle event
- ✓ setPriceOracle is alias for _setPriceOracle
- ✓ Should revert on same values

_setComptrollerLens

- ✓ fails **if** not called by admin
- ✓ should fire an event
- ✓ should revert on same value

_setCloseFactor

- ✓ fails **if** not called by admin
- ✓ should revert on same values
- ✓ fails **if** factor is set out of range

_setCollateralFactor

- ✓ fails **if** asset is **not** listed
- ✓ fails **if** factor is set without an underlying price
- ✓ succeeds **and** sets market
- ✓ succeeds **and** sets market using alias

_setForcedLiquidation

- ✓ fails **if** asset is **not** listed
- ✓ fails **if** ACM does **not** allow the call
- ✓ sets forced liquidation
- ✓ should alias setForcedLiquidation to _setForcedLiquidation
- ✓ sets forced liquidation for VAI, even though it is **not** a listed market
- ✓ emits IsForcedLiquidationEnabledUpdated event

_setForcedLiquidationForUser

- ✓ fails **if** asset is **not** listed
- ✓ fails **if** ACM does **not** allow the call
- ✓ sets forced liquidation for user
- ✓ sets forced liquidation for VAI, even though it is **not** a listed market (44ms)
- ✓ emits IsForcedLiquidationEnabledForUserUpdated event

_supportMarket

- ✓ fails **if** asset is **not** a VToken
- ✓ succeeds **and** sets market
- ✓ cannot list a market a second time (52ms)
- ✓ can list two different markets (86ms)

updateDelegate

- ✓ should revert when zero address is passed
- ✓ should revert when approval status is already set to the requested value
- ✓ should emit event on success

enterMarketBehalf

- ✓ should allow user to enter market for themselves
- ✓ should allow approved delegate to enter market on behalf of user
- ✓ should revert when unapproved delegate tries to enter market

Hooks

mintAllowed

- ✓ allows minting **if** cap is **not** reached
- ✓ reverts **if** supply cap reached
- ✓ reverts **if** market is **not** listed

redeemVerify

- ✓ should allow you to redeem 0 underlying for 0 tokens
- ✓ should allow you to redeem 5 underlying for 5 tokens
- ✓ should **not** allow you to redeem 5 underlying for 0 tokens

liquidateBorrowAllowed

Forced liquidations enabled for user

- ✓ enables forced liquidation for user
- ✓ reverts **if** borrowed market is **not** listed (65ms)
- ✓ reverts **if** collateral market is **not** listed
- ✓ does **not** revert **if** borrowed vToken is VAIController (70ms)
- ✓ allows liquidations without shortfall
- ✓ allows to repay 100% of the borrow
- ✓ fails with TOO_MUCH_REPAY **if** trying to repay > borrowed amount
- ✓ checks the shortfall **if** IsForcedLiquidationEnabledForUser is set back to false (53ms)

Forced liquidations enabled for entire market

- ✓ reverts **if** borrowed market is **not** listed
- ✓ reverts **if** collateral market is **not** listed

- ✓ does **not** revert **if** borrowed vToken is VAIController (53ms)
- ✓ allows liquidations without shortfall
- ✓ allows to repay 100% of the borrow
- ✓ fails with TOO_MUCH_REPAY **if** trying to repay > borrowed amount
- ✓ checks the shortfall **if** isForcedLiquidationEnabled is set back to false (40ms)

Forced liquidations disabled

- ✓ reverts **if** borrowed market is **not** listed (48ms)
- ✓ reverts **if** collateral market is **not** listed (61ms)
- ✓ does **not** revert **if** borrowed vToken is VAIController (74ms)
- ✓ fails **if** borrower has 0 shortfall
- ✓ succeeds **if** borrower has nonzero shortfall

borrow

- ✓ allows borrowing **if** cap is **not** reached (60ms)
- ✓ reverts borrowing **if** borrow cap is reached (60ms)
- ✓ reverts borrowing **if** borrow cap is 0 (65ms)

0x7dB92006670a82b4522eC36a5f31053Fac727892

- ✓ getBorrowingPower is an alias for getAccountLiquidity

E-Mode Pool

createPool

- ✓ reverts **if** label is empty
- ✓ should increment poolId **and** stores label

addPoolMarkets

- ✓ reverts **if** array lengths mismatch
- ✓ reverts **if** trying to **add** to core pool (poolId 0)
- ✓ reverts **if** pool does **not** exist
- ✓ reverts **if** market Already exist in the pool
- ✓ reverts **if** market **not** listed in core pool
- ✓ should **add** multiple markets (101ms)

setIsBorrowAllowed

- ✓ reverts **if** pool does **not** exist
- ✓ reverts **if** market is **not** listed in the pool
- ✓ should **return** silently **if** borrowAllowed is already set to desired value
- ✓ should update borrowAllowed **and** emits event (53ms)

setCollateralFactor for specific poolId

- ✓ reverts **if** pool does **not** exist
- ✓ reverts **if** market is **not** listed in the pool
- ✓ reverts on invalid parameter bounds
- ✓ should update collateral factor **and** liquidation threshold **and** emits event

setLiquidationIncentive with poolId

- ✓ reverts **if** pool does **not** exist
- ✓ reverts **if** market is **not** listed in the pool
- ✓ reverts on invalid parameter bounds
- ✓ should update liquidation incentive **and** emits event

removePoolMarket

- ✓ reverts **if** pool does **not** exist
- ✓ reverts **if** market is **not** listed in the pool
- ✓ removes the market **and** emits event (61ms)
- ✓ should delete pool vTokens **array** if last market removed

poolEnter

- ✓ reverts **if** entering the wrong pool
- ✓ reverts **if** entering the same pool
- ✓ reverts **if** user has invalid pool borrows
- ✓ should emit PoolSelected on successful pool switch

effective risk params

- ✓ should **return** emode params **if** market included in the emode category, else falls back to core (92ms)

setAllowCorePoolFallback

- ✓ should update the AllowCorePoolFallback
- ✓ should use risk factors as zero **if** market **not** configured in the pool
- ✓ should use core pool risk factors **if** market **not** configured in the pool **and**

allowCorePoolFallback is true (39ms)

Pool isActive Status

- ✓ reverts **if** pool does **not** exist
- ✓ reverts **if** tries to set for core pool
- ✓ should **return** silently **if** isActive is already set to desired value
- ✓ should update isActive **and** emits event

setPoolLabel

- ✓ should update the pool label

Market Getters

- ✓ returns correct key for core pool
- ✓ returns correct core pool market info using markets()
- ✓ returns correct pool market info using poolMarkets()

- ✓ returns all the markets of the specific pool (100ms)

Comptroller

- ✓ Revert on **check** for the function selector
- ✓ Add Facet **and** function selectors to proxy (64ms)
- ✓ Get all facet function selectors by facet address
- ✓ Get facet position by facet address
- ✓ Get all facet addresses
- ✓ Get all facets address **and** their selectors
- ✓ Get facet address **and** position by function selector
- ✓ Remove function selector from facet mapping
- ✓ Replace the function from facet mapping (65ms)
- ✓ Remove all functions (39ms)

FlashLoan

FlashLoan Multi-Assets

- ✓ Should revert flash loan when paused **system**-wide
- ✓ Should revert **if** flashLoan is **not** enabled
- ✓ Should revert **if** the user is zero address
- ✓ Should revert when onBehalf is address zero
- ✓ Should revert when too many assets are requested (90ms)
- ✓ Should revert **if** the market is **not** listed
- ✓ Should revert **if** contract is **not** whitelisted (38ms)
- ✓ Should revert **if** array params are unequal
- ✓ should revert when requested flash loan amount is zero (65ms)
- ✓ Should revert **if** receiver's executeOperation returns false (121ms)
- ✓ Should emit FlashLoanPauseChanged event when pause status changes
- ✓ Should **not** emit event when setting same pause status
- ✓ User has **not** supplied in venus - Should **not** create debt position **if** receiver repays full amount + fee (173ms)
- ✓ User has **not** supplied in venus - should revert **if** repayment is insufficient (116ms)
- ✓ User has supplied in venus - Should **not** create debt position **if** repays full amount + fee (263ms)
- ✓ User has supplied in venus - Should create debt position **if** repays less than required (542ms)
- ✓ User has **not** enough supply in Venus **and** repays lesser amount (should revert) (186ms)
- ✓ Should revert with NotEnoughRepayment when repayment is less than total fee (173ms)

Comptroller

liquidateCalculateAmountSeize

- ✓ fails **if** borrowed asset price is 0
- ✓ fails **if** collateral asset price is 0
- ✓ fails **if** the repayAmount causes overflow
- ✓ fails **if** the borrowed asset price causes overflow
- ✓ reverts **if** it fails to calculate the exchange rate
- ✓ returns the correct value for
1000000000000000000,1000000000000000000,1000000000000000000,1000000000000000000,1000000000000000000
- ✓ returns the correct value for
2000000000000000000,1000000000000000000,1000000000000000000,1000000000000000000,1000000000000000000
- ✓ returns the correct value for
2000000000000000000,2000000000000000000,1420000000000000000,1300000000000000000,2450000000000000000
- ✓ returns the correct value for
2789000000000000000,5230480842000000000,7713200000000000000,1300000000000000000,1.000245e+22
- ✓ returns the correct value for
7.009232529961056e+24,2.5278726317240445e+24,2.6177112093242585e+23,1179713989619784000,7.790468414639561e+24
- ✓ returns the correct value for
4.813685552195249e+24,1.7596024792088017e+24,7.754774767469792e+24,1217941672563436800,6.541621752053618e+24

ComptrollerMock

_setActionsPaused

- ✓ reverts **if** the market is **not** listed
- ✓ does nothing **if** the actions list is empty
- ✓ does nothing **if** the markets list is empty
- ✓ can pause one action on several markets
- ✓ can pause several actions on one market
- ✓ can pause **and** unpause several actions on several markets (93ms)

RepayBorrow Capping Logic Tests

Direct repayBorrow Tests

- ✓ Should cap repayment to actual debt when repayAmount > debt (67ms)
- ✓ Should work normally when repayAmount < debt (68ms)
- ✓ Should work with type(uint256).max for full repayment (71ms)

repayBorrowBehalf Tests

- ✓ Should cap repayment when paying for another user (66ms)

Liquidation Impact Tests

- ✓ Should cap liquidation repayment to borrower's actual debt (102ms)

Mathematical Safety Tests

- ✓ Should prevent underflow in debt calculations (66ms)
- ✓ Should handle edge case where debt is exactly zero (123ms)

MoveDebtDelegate

setBorrowAllowed

- ✓ fails `if` called by a non-owner
- ✓ fails `if` called with zero address for `vTokenToBorrow`
- ✓ sets `borrowAllowed` to the specified value
- ✓ emits an event
- ✓ does `not` emit an event `if` no-op

setRepaymentAllowed

- ✓ fails `if` called by a non-owner
- ✓ fails `if` called with zero address for `vTokenToRepay`
- ✓ sets `borrowAllowed` to the specified value
- ✓ emits an event
- ✓ does `not` emit an event `if` no-op

moveDebt

- ✓ fails `if` called with a token that is `not` allowed to be borrowed
- ✓ fails `if` called with a token that is `not` allowed to be repaid
- ✓ fails `if` called with a borrower who is `not` in the repayment allowlist
- ✓ succeeds `if` repayments are allowed for `ANY_USER` (81ms)
- ✓ fails `if` comptrollers don't match (41ms)
- ✓ fails `if` `repayBorrowBehalf` returns a non-zero error code
- ✓ fails `if` `borrowBehalf` returns a non-zero error code (67ms)
- ✓ transfers `repayAmount` of `vTokenToRepay.underlying()` from the sender (77ms)
- ✓ approves `vToken` to transfer money from the contract (87ms)
- ✓ calls `repayBorrowBehalf` after transferring the underlying to self (86ms)
- ✓ converts the amounts using the oracle exchange rates (87ms)
- ✓ uses the actually repaid amount rather than specified amount (102ms)
- ✓ transfers the actually borrowed amount to the owner (94ms)

sweepTokens

- ✓ fails `if` called by a non-owner
- ✓ transfers the full balance to the owner

assetListTest

swapDebt

- ✓ fails `if` called by a non-owner
- ✓ fails `if` comptrollers don't match (64ms)
- ✓ fails `if` `repayBorrowBehalf` returns a non-zero error code (52ms)
- ✓ fails `if` `borrowBehalf` returns a non-zero error code (96ms)
- ✓ transfers `repayAmount` of underlying from the sender (108ms)
- ✓ approves `vToken` to transfer money from the contract (94ms)
- ✓ calls `repayBorrowBehalf` after transferring the underlying to self (107ms)
- ✓ converts the amounts using the oracle exchange rates (98ms)
- ✓ uses the actually repaid amount rather than specified amount (105ms)
- ✓ transfers the actually borrowed amount to the owner (97ms)

sweepTokens

- ✓ fails `if` called by a non-owner
- ✓ transfers the full balance to the owner

Evil Token test

Duplicate definition of Log (Log(string,address), Log(string,uint256))

Duplicate definition of Log (Log(string,address), Log(string,uint256))

Duplicate definition of Log (Log(string,address), Log(string,uint256))

- ✓ Check the updated `vToken` states after transfer out (752ms)

BUSDLiquidator

setLiquidatorShare

- ✓ should set liquidator share
- ✓ should emit `NewLiquidatorShare` event
- ✓ should revert `if` caller is `not` owner
- ✓ should revert `if` new liquidator share is `> MANTISA_ONE`

liquidateEntireBorrow

- ✓ should repay entire borrow (600ms)
- ✓ should seize collateral `and` split correctly between liquidator `and` treasury (715ms)

liquidateBorrow

- ✓ should repay a part of the borrow (653ms)

- ✓ should seize collateral correctly for partial repay **and** split between liquidator **and** treasury (738ms)

TokenRedeemer

redeemAndTransfer

- ✓ should fail **if** called by a non-owner
- ✓ should fail **if** redeem fails (51ms)
- ✓ should succeed with zero amount (95ms)
- ✓ should redeem all vTokens (158ms)
- ✓ should transfer all underlying to the receiver (143ms)

redeemUnderlyingAndTransfer

- ✓ should fail **if** called by a non-owner
- ✓ should revert **if** redeemer does **not** have vToken balance (79ms)
- ✓ should redeem **and** transfer successfully (208ms)

redeemUnderlyingAndRepayBorrowBehalf

- ✓ should revert **if** redeemer does **not** have vToken balance (53ms)
- ✓ should redeem **and** repay successfully (478ms)

redeemAndBatchRepay

Generic

- ✓ fails **if** called by a non-owner

Full repayment

Native asset

- ✓ redeems just the required amount of vTokens (251ms)
- ✓ repays all borrows in full (275ms)
- ✓ transfers the excess vTokens to the receiver (277ms)
- ✓ transfers the excess BNB to the receiver (314ms)

Tokens

- ✓ redeems just the required amount of vTokens (313ms)
- ✓ repays up to specified caps (330ms)
- ✓ repays all borrows in full (343ms)
- ✓ transfers the excess vTokens to the receiver (326ms)
- ✓ transfers the excess underlying to the receiver (358ms)

Partial repayment

Native asset

- ✓ redeems all available vTokens, up to 1 vToken wei (214ms)
- ✓ repays the three borrows: [in full, partially, no repayment] (252ms)
- ✓ uses the excess BNB to repay the debt in full (319ms)
- ✓ does **not** keep any vBNB **or** BNB balance (250ms)

Tokens

- ✓ redeems all available vTokens, up to 1 vToken wei (297ms)
- ✓ repays the three borrows: [in full, partially, no repayment] (314ms)
- ✓ uses the excess underlying to repay the debt in full (428ms)
- ✓ does **not** keep any vToken **or** underlying balance (364ms)

batchRepayVAI

- ✓ fails **if** called by a non-owner
- ✓ repays one borrow successfully (280ms)
- ✓ repays multiple borrows successfully **and** transfers refund to treasury (704ms)
- ✓ repays up to caps (658ms)
- ✓ partially repays borrows **if** insufficient VAI (558ms)
- ✓ can repay small amounts without failure (857ms)

sweepTokens

- ✓ fails **if** called by a non-owner
- ✓ sweeps tokens to destination **if** called by owner (52ms)
- ✓ sweeps native asset to destination

Two Kinks Interest Rate Model Tests

- ✓ Utilization rate: borrows is zero
- ✓ Utilization rate
- ✓ Borrow Rate: below kink1 utilization (40ms)
- ✓ Borrow Rate: above kink1 **and** below kink2 utilization (47ms)
- ✓ Borrow Rate: above kink2 utilization (50ms)
- ✓ Borrow Rate: above kink2 utilization **and** negative multipliers (80ms)
- ✓ Supply Rate

VenusLens: Rewards Summary

- ✓ Should get summary for all markets (266ms)

Liquidator

splitLiquidationIncentive

- ✓ splits liquidationIncentive between Treasury **and** Liquidator with correct amounts

distributeLiquidationIncentive

- ✓ distributes the liquidationIncentive between Treasury **and** Liquidator with correct amounts (62ms)

- ✓ reverts **if** transfer to liquidator fails
- ✓ reverts **if** underlying transfer to protocol share reserves fails (50ms)

Liquidator

liquidateBorrow

liquidating BEP-20 debt

network block skew detected; skipping block events (emitted=2993 blockNumber4000)

network block skew detected; skipping block events (emitted=2993 blockNumber4000)

network block skew detected; skipping block events (emitted=2993 blockNumber4000)

- ✓ fails **if** borrower is zero address
- ✓ fails **if** some BNB is sent along with the transaction (47ms)
- ✓ transfers the seized collateral to liquidator **and** protocolShareReserve (137ms)
- ✓ transfers tokens from the liquidator (191ms)

network block skew detected; skipping block events (emitted=2995 blockNumber4004)

network block skew detected; skipping block events (emitted=2995 blockNumber4004)

- ✓ approves the borrowed VToken to spend underlying (181ms)
- ✓ calls liquidateBorrow on borrowed VToken (169ms)
- ✓ emits LiquidateBorrowedTokens event (166ms)

liquidating VAI debt

- ✓ transfers VAI from the liquidator (157ms)
- ✓ approves VAIController to spend VAI (129ms)
- ✓ calls liquidateVAI on VAIController (113ms)

liquidating BNB debt

- ✓ fails **if** msg.value is **not** equal to repayment amount (85ms)
- ✓ transfers BNB from the liquidator (94ms)
- ✓ calls liquidateBorrow on VBNB (95ms)
- forwards BNB to VBNB contract

setTreasuryPercent

- ✓ updates treasury percent in storage
- ✓ fails when permission is **not** granted
- ✓ fails when the percentage is too high
- ✓ uses the **new** treasury percent during distributions (151ms)

Force VAI Liquidation

- ✓ Should able to liquidate any token when VAI debt is lower than minLiquidatableVAI (84ms)
- ✓ Should **not** able to liquidate any token when VAI debt is greater than minLiquidatableVAI
- ✓ Should able to liquidate any token when VAI debt is greater than minLiquidatableVAI but forced

liquidation is enabled

- ✓ Should able to liquidate VAI token when VAI debt is greater than minLiquidatableVAI (124ms)
- ✓ Should able to liquidate any token **and** VAI token when force Liquidation is off (143ms)

Liquidator

Restricted liquidations

addToAllowlist

- ✓ fails **if** not allowed to call
- ✓ adds address to allowlist
- ✓ fails **if** already in the allowlist (39ms)
- ✓ emits LiquidationPermissionGranted event

removeFromAllowlist

- ✓ fails **if** not allowed to call
- ✓ fails **if** not in the allowlist
- ✓ removes address from allowlist (72ms)
- ✓ emits LiquidationPermissionRevoked event (47ms)

restrictLiquidation

- ✓ fails **if** not allowed to call
- ✓ restricts liquidations for the borrower
- ✓ fails **if** already restricted (52ms)
- ✓ emits LiquidationRestricted event

unrestrictLiquidation

- ✓ fails **if** not allowed to call
- ✓ removes the restrictions for the borrower (61ms)
- ✓ fails **if** not restricted
- ✓ emits LiquidationRestricted event (43ms)

liquidateBorrow

- ✓ fails **if** the liquidation is restricted
- ✓ proceeds with the liquidation **if** the guy is allowed to (65ms)

PrimeScenario Token

setMaxLoopsLimit()

Warning: Potentially unsafe deployment of

contracts/Tokens/Prime/PrimeLiquidityProvider.sol:PrimeLiquidityProvider

You are using the ``unsafeAllow.internal-function-storage`` flag.

Internal functions are code pointers which will no longer be valid after an upgrade.
Make sure you reassign internal functions in storage variables during upgrades.

Warning: Potentially unsafe deployment of contracts/test/PrimeScenario.sol:PrimeScenario

You are using the `unsafeAllow.internal-function-storage` flag.
Internal functions are code pointers which will no longer be valid after an upgrade.
Make sure you reassign internal functions in storage variables during upgrades.

- ✓ Revert when maxLoopsLimit setter is called by non-owner
- ✓ Revert when **new** loops limit is less than old limit
- ✓ maxLoopsLimit setter success (39ms)

protocol setup

- ✓ markets added
- ✓ borrow balance
- ✓ get markets in prime

mint **and** burn

- ✓ stake **and** mint (293ms)
- ✓ stake **and** unstake (221ms)
- ✓ stake manually (268ms)

network block skew detected; skipping block events (emitted=4023 blockNumber7780068)

- ✓ burn revocable token (704ms)
- ✓ cannot burn irrevocable token (662ms)
- ✓ manually burn irrevocable token (498ms)
- ✓ issue (702ms)
- ✓ upgrade (503ms)
- ✓ stake, issue **and** unstake (920ms)

network block skew detected; skipping block events (emitted=7780068 blockNumber8644069)

- ✓ issue, stake **and** burn (853ms)

boosted yield

- ✓ calculate score (130ms)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4074 blockNumber7780067)

network block skew detected; skipping block events (emitted=4073 blockNumber7780076)

network block skew detected; skipping block events (emitted=4073 blockNumber7780076)

network block skew detected; skipping block events (emitted=4073 blockNumber7780077)

network block skew detected; skipping block events (emitted=4074 blockNumber7780080)

network block skew detected; skipping block events (emitted=4074 blockNumber7780080)

- ✓ accrue interest – prime token minted after market is added (469ms)
 - ✓ claim interest (261ms)
- update score
- ✓ **add** existing market after issuing prime tokens – update score gradually (718ms)
 - ✓ **add** existing market after issuing prime tokens – update score manually (1185ms)

PLP integration

- ✓ claim interest (399ms)
- ✓ APR Estimation (95ms)
- ✓ Hypothetical APR Estimation (269ms)

PrimeLiquidityProvider: tests

Testing all initialized values

Warning: Potentially unsafe deployment of

contracts/Tokens/Prime/PrimeLiquidityProvider.sol:PrimeLiquidityProvider

You are using the `unsafeAllow.internal-function-storage` flag.
Internal functions are code pointers which will no longer be valid after an upgrade.
Make sure you reassign internal functions in storage variables during upgrades.

- ✓ Tokens initialized
- ✓ Distribution Speed

Testing all setters

- ✓ Revert on invalid args for initializeTokens
- ✓ Revert on re-initializing token

- ✓ initializeTokens success
- ✓ pauseFundsTransfer
- ✓ resumeFundsTransfer (60ms)
- ✓ Revert on invalid args for setTokensDistributionSpeed
- ✓ Revert on non initialized token
- ✓ Revert on invalid distribution speed for setTokensDistributionSpeed (52ms)
- ✓ setTokensDistributionSpeed success with default max speed (54ms)
- ✓ setTokensDistributionSpeed success (58ms)
- ✓ setMaxTokensDistributionSpeed success
- ✓ Reverts on setting prime address same as previous
- ✓ Revert on invalid prime token address
- ✓ Revert when prime token setter is called by non-owner
- ✓ setPrimeToken success
- ✓ Revert when maxLoopsLimit setter is called by non-owner
- ✓ Revert when **new** loops limit is less than old limit
- ✓ maxLoopsLimit setter success

Accrue tokens

- ✓ Revert on non initialized token
- ✓ Accrue amount for tokenA (55ms)
- ✓ Accrue amount for multiple tokens (345ms)

Release funds to prime contract

- ✓ Revert on funds transfer Paused
- ✓ Revert on invalid caller
- ✓ Release funds success (64ms)

Sweep token

- ✓ Revert on insufficient balance
- ✓ Sweep token success (49ms)

Swap Contract

- ✓ revert **if** vToken address is **not** listed

Setter

- ✓ should reverted **if** zero address
- ✓ should reverted **if** vToken **not** listed
- ✓ setting address for VBNBToken (42ms)

Swap

- ✓ revert **if** path length is 1
- ✓ revert **if** deadline has passed
- ✓ revert **if** output amoutn is below minimum
- ✓ should be reverted **if** tokenA == tokenB
- ✓ should swap tokenA -> tokenB (44ms)
- ✓ revert **if** deadline has passed
- ✓ revert **if** address zero
- ✓ should reverted **if** first address in **not** WBNB address
- ✓ should reverted **if** output amount is below minimum
- ✓ should swap BNB -> token (46ms)
- ✓ revert **if** deadline has passed
- ✓ should swap tokenA -> tokenB at supporting fee
- ✓ should reverted **if** deadline passed
- ✓ should swap BNB -> token at supporting fee
- ✓ should swap EXact token -> BNB at supporting fee (66ms)
- ✓ should swap tokens for Exact BNB
- ✓ should swap tokens for Exact Tokens
- ✓ should swap tokens for Exact BNB
- ✓ should swap BNB for Exact Tokens

Supply

- ✓ revert **if** deadline has passed
- ✓ swap tokenA -> tokenB --> supply tokenB (88ms)
- ✓ swap BNB -> token --> supply token (83ms)
- ✓ revert **if** deadline has passed at supporting fee
- ✓ swap tokenA -> tokenB --> supply tokenB at supporting fee (92ms)
- ✓ swap BNB -> token --> supply token at supporting fee (112ms)
- ✓ swap tokenA -> exact tokenB (93ms)
- ✓ swap bnb -> exact tokenB (89ms)
- ✓ Exact tokens -> BNB **and** supply
- ✓ Exact tokens -> BNB **and** supply at supporting fee

Repay

- ✓ revert **if** deadline has passed
- ✓ swap tokenA -> tokenB --> supply tokenB (81ms)
- ✓ swap BNB -> token --> supply token (82ms)
- ✓ revert **if** deadline has passed at supporting fee
- ✓ swap tokenA -> tokenB --> reapy tokenB at supporting fee (90ms)
- ✓ swap BNB -> token --> repay token at supporting fee (93ms)

- ✓ swap tokenA -> exact tokenB (95ms)
- ✓ swap tokenA -> full debt of tokenB (91ms)
- ✓ swap bnb -> exact tokenB (84ms)
- ✓ swap bnb -> full tokenB debt (112ms)
- ✓ Exact tokens -> BNB at supporting fee (65ms)
- ✓ Exact tokens -> BNB (55ms)
- ✓ Tokens -> Exact BNB (55ms)
- ✓ Tokens -> Exact BNB **and** supply
- ✓ Tokens -> full debt of BNB

Sweep Token

- ✓ Should be reverted **if** get zero address
- ✓ Sweep ERC-20 tokens (61ms)

library function

- ✓ Quote function
- ✓ getAmoutIn function
- ✓ getAmoutout function
- ✓ getAmoutout function
- ✓ getAmoutout function

admin / _setPendingAdmin / _acceptAdmin

- admin()
 - ✓ should **return** correct admin
- pendingAdmin()
 - ✓ should **return** correct pending admin
- _setPendingAdmin()
 - ✓ should only be callable by admin
 - ✓ should properly set pending admin
 - ✓ should properly set pending admin twice
 - ✓ should emit event
- _acceptAdmin()
 - ✓ should fail when pending admin is zero
 - ✓ should fail when called by another account (e.g. root)
 - ✓ should succeed **and** set admin **and** clear pending admin
 - ✓ should emit log on success

Unitroller

- constructor**
 - ✓ sets admin to caller **and** addresses to 0
- _setPendingImplementation
 - Check caller is admin
 - ✓ emits a failure log
 - ✓ does **not** change pending implementation address
 - succeeding
 - ✓ stores pendingComptrollerImplementation with value newPendingImplementation
 - ✓ emits NewPendingImplementation event
- _acceptImplementation
 - Check caller is pendingComptrollerImplementation **and** pendingComptrollerImplementation ≠ address(0)
 - ✓ emits a failure log
 - ✓ does **not** change current implementation address
 - the brains must accept the responsibility of implementation
 - ✓ Store comptrollerImplementation with value pendingComptrollerImplementation
 - ✓ Unset pendingComptrollerImplementation
 - ✓ Emit NewImplementation(oldImplementation, newImplementation)
 - ✓ Emit NewPendingImplementation(oldPendingImplementation, 0)
 - fallback delegates to brains
 - ✓ forwards reverts
 - ✓ gets addresses
 - ✓ gets strings
 - ✓ gets bools
 - ✓ gets list of ints

CheckpointView tests (using interest rate models as data sources)

- ✓ should revert **if** dataSource1 address is zero
- ✓ should revert **if** dataSource2 address is zero
- ✓ should use old rate model before checkpoint (46ms)
- ✓ should use **new** rate model after checkpoint (45ms)
- ✓ should **return** the correct current data source

Peg Stability Module

PSM: 18 decimals

initialization

- ✓ should revert **if** contract already deployed

```

✓ should initialize successfully
reverts if init address = 0x0:
  ✓ acm
  ✓ treasury
  ✓ stableToken
reverts if fee init value is invalid
  ✓ feeIn
  ✓ feeOut
Admin functions
pause()
  ✓ should revert if not authorised
  ✓ should pause if authorised
  ✓ should revert if already paused
resume()
  ✓ should revert if not authorised
  ✓ should resume if authorised
  ✓ should revert if already resumed
setFeeIn(uint256)
  ✓ should revert if not authorised
  ✓ should revert if fee is invalid
  ✓ set the correct fee
setFeeOut(uint256)
  ✓ should revert if not authorised
  ✓ should revert if fee is invalid
  ✓ set the correct fee
setVAIMintCap(uint256)
  ✓ should revert if not authorised
  ✓ should set the correct mint cap
setVenusTreasury(uint256)
  ✓ should revert if not authorised
  ✓ should revert if zero address
  ✓ should set the treasury address
setOracle(address)
  ✓ should revert if not authorised
  ✓ should revert if oracle address is zero
  ✓ should set the oracle (47ms)
Pause logic
  ✓ should revert when paused and call swapVAIForStable(address,uint256)
  ✓ should revert when paused and call swapStableForVAI(address,uint256)
Swap functions
swapVAIForStable(address,uint256)
  ✓ should revert if receiver is zero address
  ✓ should revert if sender has insufficient VAI balance (42ms)
  ✓ should revert if VAI transfer fails (39ms)
  ✓ should revert if VAI to be burnt > vaiMinted
  should successfully perform the swap
    Fees: 10%
      ✓ stable token = 1$ (61ms)
      ✓ stable token < 1$ (56ms)
      ✓ stable token > 1$ (59ms)
    Fees: 0%
      ✓ stable token = 1$ (51ms)
      ✓ stable token < 1$ (60ms)
      ✓ stable token > 1$ (48ms)
swapStableForVAI(address,uint256)
  ✓ should revert if receiver is zero address
  ✓ should revert if VAI mint cap will be reached (57ms)
  ✓ should revert if amount after transfer is too small (48ms)
  should successfully perform the swap
    Fees: 10%
      ✓ stable token = 1$ (75ms)
      ✓ stable token > 1$ (74ms)
      ✓ stable token < 1$ (72ms)
    Fees: 0%
      ✓ stable token = 1$ (66ms)
      ✓ stable token > 1$ (69ms)
      ✓ stable token < 1$ (68ms)
PSM: 8 decimals
initialization
  ✓ should revert if contract already deployed
  ✓ should initialize successfully
reverts if init address = 0x0:

```


- ✓ acm
- ✓ treasury
- ✓ stableToken

reverts **if** fee init value is invalid

- ✓ feeIn
- ✓ feeOut

Admin functions

pause()

- ✓ should revert **if** not authorised
- ✓ should pause **if** authorised
- ✓ should revert **if** already paused

resume()

- ✓ should revert **if** not authorised
- ✓ should resume **if** authorised
- ✓ should revert **if** already resumed

setFeeIn(uint256)

- ✓ should revert **if** not authorised
- ✓ should revert **if** fee is invalid
- ✓ set the correct fee

setFeeOut(uint256)

- ✓ should revert **if** not authorised
- ✓ should revert **if** fee is invalid
- ✓ set the correct fee

setVAIMintCap(uint256)

- ✓ should revert **if** not authorised
- ✓ should set the correct mint cap

setVenusTreasury(uint256)

- ✓ should revert **if** not authorised
- ✓ should revert **if** zero address
- ✓ should set the treasury address

setOracle(address)

- ✓ should revert **if** not authorised
- ✓ should revert **if** oracle address is zero
- ✓ should set the oracle (52ms)

Pause logic

- ✓ should revert when paused **and** call swapVAIForStable(address,uint256)
- ✓ should revert when paused **and** call swapStableForVAI(address,uint256)

Swap functions

swapVAIForStable(address,uint256)

- ✓ should revert **if** receiver is zero address
- ✓ should revert **if** sender has insufficient VAI balance (47ms)
- ✓ should revert **if** VAI transfer fails (55ms)
- ✓ should revert **if** VAI to be burnt > vaiMinted (46ms)

should successfully perform the swap

Fees: 10%

- ✓ stable token = 1\$ (84ms)
- ✓ stable token < 1\$ (84ms)
- ✓ stable token > 1\$ (80ms)

Fees: 0%

- ✓ stable token = 1\$ (66ms)
- ✓ stable token < 1\$ (66ms)
- ✓ stable token > 1\$ (66ms)

swapStableForVAI(address,uint256)

- ✓ should revert **if** receiver is zero address
- ✓ should revert **if** VAI mint cap will be reached (70ms)

should successfully perform the swap

Fees: 10%

- ✓ stable token = 1\$ (89ms)
- ✓ stable token > 1\$ (98ms)
- ✓ stable token < 1\$ (92ms)

Fees: 0%

- ✓ stable token = 1\$ (81ms)
- ✓ stable token > 1\$ (89ms)
- ✓ stable token < 1\$ (89ms)

PSM: 6 decimals

initialization

- ✓ should revert **if** contract already deployed
- ✓ should initialize successfully

reverts **if** init address = 0x0:

- ✓ acm
- ✓ treasury
- ✓ stableToken

```

    reverts if fee init value is invalid
    ✓ feeIn
    ✓ feeOut
Admin functions
pause()
    ✓ should revert if not authorised
    ✓ should pause if authorised
    ✓ should revert if already paused
resume()
    ✓ should revert if not authorised
    ✓ should resume if authorised
    ✓ should revert if already resumed
setFeeIn(uint256)
    ✓ should revert if not authorised
    ✓ should revert if fee is invalid
    ✓ set the correct fee
setFeeOut(uint256)
    ✓ should revert if not authorised
    ✓ should revert if fee is invalid
    ✓ set the correct fee
setVAIMintCap(uint256)
    ✓ should revert if not authorised
    ✓ should set the correct mint cap
setVenusTreasury(uint256)
    ✓ should revert if not authorised
    ✓ should revert if zero address
    ✓ should set the treasury address
setOracle(address)
    ✓ should revert if not authorised
    ✓ should revert if oracle address is zero
    ✓ should set the oracle (54ms)
Pause logic
    ✓ should revert when paused and call swapVAIForStable(address,uint256)
    ✓ should revert when paused and call swapStableForVAI(address,uint256)
Swap functions
swapVAIForStable(address,uint256)
    ✓ should revert if receiver is zero address
    ✓ should revert if sender has insufficient VAI balance (51ms)
    ✓ should revert if VAI transfer fails (74ms)
    ✓ should revert if VAI to be burnt > vaiMinted (58ms)
    should successfully perform the swap
    Fees: 10%
        ✓ stable token = 1$ (100ms)
        ✓ stable token < 1$ (98ms)
        ✓ stable token > 1$ (94ms)
    Fees: 0%
        ✓ stable token = 1$ (81ms)
        ✓ stable token < 1$ (84ms)
        ✓ stable token > 1$ (85ms)
swapStableForVAI(address,uint256)
    ✓ should revert if receiver is zero address
    ✓ should revert if VAI mint cap will be reached (80ms)
    should successfully perform the swap
    Fees: 10%
        ✓ stable token = 1$ (108ms)
        ✓ stable token > 1$ (109ms)
        ✓ stable token < 1$ (107ms)
    Fees: 0%
        ✓ stable token = 1$ (98ms)
        ✓ stable token > 1$ (101ms)
        ✓ stable token < 1$ (100ms)

VAIController
    ✓ check wallet usdt balance
    #getMintableVAI
        ✓ oracle
        ✓ getAssetsIn
        ✓ getAccountSnapshot
        ✓ getUnderlyingPrice (42ms)
        ✓ getComtroller
        ✓ success (148ms)
    #mintVAI

```

```
✓ success (288ms)
✓ fails if there's not enough collateral (214ms)
✓ fails if minting beyond mint cap (347ms)
✓ fails if can't set the minted amount in comptroller (235ms)
✓ puts previously accrued interest to pastInterest (592ms)
#repayVAI
✓ reverts if the protocol is paused
✓ success for zero rate (174ms)
✓ success for 1.2 rate repay all (246ms)
✓ success for 1.2 rate repay half (235ms)
✓ fails if can't set the new minted amount in comptroller (149ms)
#repayVAIBehalf
✓ reverts if called with borrower = zero address
✓ reverts if the protocol is paused (38ms)
✓ success for zero rate (173ms)
✓ success for 1.2 rate repay all (241ms)
✓ success for 1.2 rate repay half (234ms)
#getHypotheticalAccountLiquidity
✓ success for zero rate 0.9 vusdt collateralFactor (254ms)
✓ success for 1.2 rate 0.9 vusdt collateralFactor (330ms)
#liquidateVAI
✓ liquidationIncentiveMantissa
✓ reverts if the protocol is paused
network block skew detected; skipping block events (emitted=8644069 blockNumber100000000)
✓ success for zero rate 0.2 vusdt collateralFactor (865ms)
network block skew detected; skipping block events (emitted=7780435 blockNumber100000000)
network block skew detected; skipping block events (emitted=7780435 blockNumber100000000)
network block skew detected; skipping block events (emitted=7780435 blockNumber100000000)
network block skew detected; skipping block events (emitted=7780435 blockNumber100000000)
network block skew detected; skipping block events (emitted=7780435 blockNumber100000000)
network block skew detected; skipping block events (emitted=7780435 blockNumber100000000)
network block skew detected; skipping block events (emitted=7780435 blockNumber100000000)
network block skew detected; skipping block events (emitted=7780435 blockNumber100000000)
network block skew detected; skipping block events (emitted=7780437 blockNumber100000000)
✓ success for 1.2 rate 0.3 vusdt collateralFactor (877ms)
#getVAIRepayRate
✓ success for zero baseRate
✓ success for baseRate 0.1 floatRate 0.1 vaiPirce 1e18 (136ms)
✓ success for baseRate 0.1 floatRate 0.1 vaiPirce 0.5 * 1e18 (134ms)
#getVAIRepayAmount
✓ reverts if the protocol is paused (53ms)
✓ success for zero rate
✓ success for baseRate 0.1 floatRate 0.1 vaiPirce 1e18 (155ms)
✓ success for baseRate 0.1 floatRate 0.1 vaiPirce 0.5 * 1e18 (174ms)
#getVAICalculateRepayAmount
✓ success for zero rate (47ms)
✓ success for baseRate 0.1 floatRate 0.1 vaiPirce 1e18 (271ms)
✓ success for baseRate 0.1 floatRate 0.1 vaiPirce 0.5 * 1e18 (284ms)
#getMintableVAI
✓ include current interest when calculating mintable VAI (270ms)
#accrueVAIInterest
✓ success for called once (104ms)
✓ success for called twice (150ms)
#setBaseRate
✓ fails if access control does not allow the call
✓ emits NewVAIBaseRate event
✓ sets new base rate in storage
#setFloatRate
✓ fails if access control does not allow the call
✓ emits NewVAIFloatRate event
✓ sets new float rate in storage
#setMintCap
✓ fails if access control does not allow the call
✓ emits NewVAIMintCap event
✓ sets new mint cap in storage
#setReceiver
✓ fails if called by a non-admin
✓ reverts if the receiver is zero address
✓ emits NewVAIRceiver event
✓ sets VAI receiver address in storage
#setAccessControl
```

- ✓ reverts `if` called by non-admin
- ✓ reverts `if` ACM is zero address
- ✓ emits NewAccessControl event (52ms)
- ✓ sets ACM address in storage (42ms)

#prime

- ✓ prime integration (1659ms)

VAIVault

- ✓ claim reward (547ms)
- setVenusInfo
- ✓ fails `if` called by a non-admin
 - ✓ fails `if` XVS address is zero
 - ✓ fails `if` VAI address is zero
 - ✓ disallows configuring tokens twice

VRTVault

unit tests

setLastAccruingBlock

- ✓ fails `if` ACM disallows the call
- ✓ fails `if` trying to set lastAccruingBlock to some absurdly high value
- ✓ fails `if` lastAccruingBlock has passed (59ms)
- ✓ fails `if` trying to set lastAccruingBlock to some past block
- ✓ fails `if` trying to set lastAccruingBlock to the current block
- ✓ correctly sets lastAccruingBlock to some future block (45ms)
- ✓ can `move` lastAccruingBlock to a later block (75ms)
- ✓ can `move` lastAccruingBlock to an earlier block (76ms)
- ✓ fails `if` trying to `move` lastAccruingBlock to a block in the past (62ms)

scenario

- ✓ deposit (98ms)
- ✓ should claim reward (62ms)
- ✓ should `not` claim reward after certain block (100ms)

VToken

_setReserveFactorFresh

network block skew detected; skipping block events (emitted=7780437 blockNumber7790568)

network block skew detected; skipping block events (emitted=7780437 blockNumber7790568)

network block skew detected; skipping block events (emitted=7780437 blockNumber7790568)

- ✓ rejects change by non-admin

network block skew detected; skipping block events (emitted=7780440 blockNumber7790615)

network block skew detected; skipping block events (emitted=7780440 blockNumber7790615)

network block skew detected; skipping block events (emitted=7780440 blockNumber7790615)

network block skew detected; skipping block events (emitted=7780440 blockNumber7790615)

- ✓ rejects change `if` market `not` fresh
- ✓ rejects newReserveFactor that descales to 1 (48ms)
- ✓ accepts newReserveFactor in valid range `and` emits log (63ms)
- ✓ accepts a change back to zero (118ms)

_setReserveFactor

- ✓ emits a reserve factor failure `if` interest accrual fails (87ms)
- ✓ returns error from setReserveFactorFresh without emitting any extra logs (62ms)
- ✓ returns success from setReserveFactorFresh (96ms)

_reduceReservesFresh

- ✓ fails `if` called by non-admin (46ms)
- ✓ fails `if` market `not` fresh (45ms)
- ✓ fails `if` amount exceeds available cash (270ms)
- ✓ `if` there isn't enough cash, reduces with available cash (146ms)
- ✓ increases admin balance `and` reduces reserves on success (188ms)

_reduceReserves

- ✓ emits a reserve-reduction failure `if` interest accrual fails (69ms)
- ✓ returns error from _reduceReservesFresh without emitting any extra logs (151ms)
- ✓ returns success code from _reduceReservesFresh `and` reduces the correct amount (142ms)

VToken

sweepTokenAndSync

sync (transferAmount = 0)

- ✓ sets internalCash to actual token balance (85ms)
- ✓ emits CashSynced event (63ms)
- ✓ reverts when called by non-admin
- ✓ is idempotent (136ms)
- ✓ works when balance is zero (58ms)

sweep (transferAmount > 0)

- ✓ recovers excess tokens `and` emits TokenSwept + CashSynced (108ms)
- ✓ internalCash unchanged after sweeping exact excess (92ms)

- ✓ reverts when transferAmount exceeds balance (61ms)
- ✓ reverts when called by non-admin (44ms)
- ✓ sweeps exact excess when multiple donations occur (120ms)
- ✓ can sweep again after a **new** donation (148ms)

XVSVault

setXvsStore

- ✓ fails **if** XVS is a zero address
- ✓ fails **if** XVSSStore is a zero address
- ✓ fails **if** the vault is already initialized

add

- ✓ reverts **if** ACM does **not** allow the call
- ✓ reverts **if** xvsStore is **not** set
- ✓ reverts **if** a pool with this (staked token, reward token) combination already exists (46ms)
- ✓ reverts **if** staked token exists in another pool
- ✓ reverts **if** reward token is a zero address
- ✓ reverts **if** staked token is a zero address
- ✓ reverts **if** alloc points parameter is zero
- ✓ emits PoolAdded event (44ms)
- ✓ adds a second pool to an existing rewardToken (61ms)
- ✓ sets pool info (65ms)
- ✓ configures reward token in XVSSStore (60ms)

set

- ✓ reverts **if** ACM does **not** allow the call
- ✓ reverts **if** pool is **not** found
- ✓ reverts **if** total alloc points after the call is zero (44ms)
- ✓ succeeds **if** the pool alloc points is zero but total alloc points is nonzero (177ms)
- ✓ emits PoolUpdated event (48ms)

setRewardAmountPerBlockOrSecond

- ✓ reverts **if** ACM does **not** allow the call
- ✓ reverts **if** the token is **not** configured in XVSSStore (58ms)
- ✓ emits RewardAmountPerBlockUpdated event (64ms)
- ✓ updates reward amount per block (73ms)

setWithdrawalLockingPeriod

- ✓ reverts **if** ACM does **not** allow the call
- ✓ reverts **if** pool does **not** exist
- ✓ reverts **if** the lock period is 0
- ✓ reverts **if** the lock period is absurdly high
- ✓ emits WithdrawalLockingPeriodUpdated event (66ms)
- ✓ updates lock period (100ms)

pendingReward

- ✓ includes the old withdrawal requests in the rewards computation (179ms)
- ✓ excludes the **new** withdrawal requests from the rewards computation (246ms)

deposit

- ✓ reverts **if** the vault is paused (45ms)
- ✓ reverts **if** pool does **not** exist
- ✓ transfers pool token to the vault (80ms)
- ✓ updates user's balance (78ms)
- ✓ fails **if** there's a pre-upgrade withdrawal request (116ms)
- ✓ succeeds **if** the pre-upgrade withdrawal request has been executed (379ms)
- ✓ uses the safe `_transferReward` under the hood (231ms)

executeWithdrawal

- ✓ fails **if** the vault is paused (45ms)
- ✓ only transfers the requested amount for post-upgrade requests (249ms)
- ✓ handles pre-upgrade withdrawal requests (242ms)
- ✓ handles pre-upgrade **and** post-upgrade withdrawal requests (392ms)

requestWithdrawal

- ✓ fails **if** the vault is paused (48ms)
- ✓ transfers rewards to the user (211ms)
- ✓ uses the safe `_transferReward` under the hood (210ms)
- ✓ fails **if** there's a pre-upgrade withdrawal request (99ms)

claim

- ✓ fails **if** there's a pre-upgrade withdrawal request (52ms)

network block skew detected; skipping block events (emitted=7791252 blockNumber7792760)

network block skew detected; skipping block events (emitted=7791252 blockNumber7792760)

network block skew detected; skipping block events (emitted=7791252 blockNumber7792760)

network block skew detected; skipping block events (emitted=7791252 blockNumber7792760)

- ✓ succeeds **if** the pre-upgrade withdrawal request has been executed (249ms)
- ✓ excludes pending withdrawals from the user's shares (304ms)
- ✓ correctly accounts for updates in reward per block (186ms)
- ✓ uses the safe `_transferReward` under the hood (127ms)

_transferReward

- ✓ sends the available funds to the user (108ms)
- ✓ emits VaultDebtUpdated event **if** vault debt is updated (75ms)
- ✓ does **not** emit VaultDebtUpdated event **if** vault debt is **not** updated (91ms)
- ✓ records the pending transfer (91ms)
- ✓ records several pending transfers (194ms)
- ✓ sends out the pending transfers in addition to reward **if** full amount <= funds available (305ms)
- ✓ sends a part of the pending transfers **and** reward **if** full amount > funds available (303ms)

pendingWithdrawalsBeforeUpgrade

- ✓ returns zero **if** there were no pending withdrawals
- ✓ returns zero **if** there is only a **new-style** pending withdrawal (117ms)
- ✓ returns the requested amount **if** there is an old-style pending withdrawal
- ✓ returns the total requested amount **if** there are multiple old-style pending withdrawals (55ms)
- ✓ returns zero **if** the pending withdrawal was executed (133ms)

Scenarios

- ✓ works correctly with multiple claim, deposit, **and** withdrawal requests (925ms)

Prime Token

mint **and** burn

network block skew detected; skipping block events (emitted=7791259 blockNumber7795068)
network block skew detected; skipping block events (emitted=7791259 blockNumber7795068)
network block skew detected; skipping block events (emitted=7791259 blockNumber7795068)
network block skew detected; skipping block events (emitted=7792760 blockNumber7795076)
network block skew detected; skipping block events (emitted=7792760 blockNumber7795076)
network block skew detected; skipping block events (emitted=7792760 blockNumber7795076)
network block skew detected; skipping block events (emitted=7792760 blockNumber7795076)

Warning: Potentially unsafe deployment of

contracts/Tokens/Prime/PrimeLiquidityProvider.sol:PrimeLiquidityProvider

You are using the `unsafeAllow.internal-function-storage` flag.

Internal functions are code pointers which will no longer be valid after an upgrade.

Make sure you reassign internal functions in storage variables during upgrades.

Warning: Potentially unsafe deployment of contracts/Tokens/Prime/Prime.sol:Prime

You are using the `unsafeAllow.internal-function-storage` flag.

Internal functions are code pointers which will no longer be valid after an upgrade.

Make sure you reassign internal functions in storage variables during upgrades.

- ✓ should alias setPrimeToken to _setPrimeToken
- ✓ stake **and** mint (1064ms)

network block skew detected; skipping block events (emitted=7795076 blockNumber15571125)
network block skew detected; skipping block events (emitted=7795076 blockNumber15571125)
network block skew detected; skipping block events (emitted=7795076 blockNumber15571125)
network block skew detected; skipping block events (emitted=7795076 blockNumber15571125)

- ✓ burn revocable token (2516ms)
- ✓ cannot burn irrevocable token (2402ms)
- ✓ issue **and** stake token concurrently (1764ms)

boosted yield

network block skew detected; skipping block events (emitted=7795123 blockNumber15571138)
network block skew detected; skipping block events (emitted=7795123 blockNumber15571138)
network block skew detected; skipping block events (emitted=7795123 blockNumber15571138)
network block skew detected; skipping block events (emitted=15571138 blockNumber23347154)
network block skew detected; skipping block events (emitted=15571138 blockNumber23347154)
network block skew detected; skipping block events (emitted=15571138 blockNumber23347154)

- ✓ claim interest for multiple users (5964ms)

798 passing (7m)

1 pending

Code Coverage

Coverage was obtained by running `npx hardhat coverage`.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts/	100	100	100	100	
InterfacesV8.sol	100	100	100	100	
contracts/Admin/	90.48	40.91	85.71	88.46	
VBNBAAdmin.sol	90.48	40.91	85.71	88.46	71,72,73
VBNBAAdminStorage.sol	100	100	100	100	
contracts/Comptroller/	100	90	100	100	
ComptrollerInterface.sol	100	100	100	100	
ComptrollerLensInterface.sol	100	100	100	100	
ComptrollerStorage.sol	100	100	100	100	
Unitroller.sol	100	90	100	100	
contracts/Comptroller/Diamond/	97.26	59.09	100	95.35	
Diamond.sol	97.26	59.09	100	95.35	109,228,229,230
DiamondConsolidated.sol	100	100	100	100	
contracts/Comptroller/Diamond/facets/	81.54	68.91	86.62	81.6	
FacetBase.sol	65.31	55.88	88.89	62.26	... 133,222,235
FlashLoanFacet.sol	100	83.33	100	94.64	73,105,311
MarketFacet.sol	96.27	71.7	91.18	95.83	... 466,467,616
PolicyFacet.sol	85.82	71.62	100	85	... 417,540,543
RewardFacet.sol	1.67	0	10	1.52	... 243,244,255
SetterFacet.sol	89	78.57	90	89.03	... 639,683,731
XVSRewardsHelper.sol	94.12	80	100	95.45	79,108
contracts/Comptroller/Diamond/interfaces/	100	100	100	100	
IDiamondCut.sol	100	100	100	100	
IFacetBase.sol	100	100	100	100	
IFlashLoanFacet.sol	100	100	100	100	

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
IMarketFacet.sol	100	100	100	100	
IPolicyFacet.sol	100	100	100	100	
IRewardFacet.sol	100	100	100	100	
ISetterFacet.sol	100	100	100	100	
contracts/Comptroller/Types/	100	100	100	100	
PoolMarketId.sol	100	100	100	100	
contracts/Comptroller/legacy/	100	100	100	100	
ComptrollerInterfaceR1.sol	100	100	100	100	
ComptrollerLensInterfaceR1.sol	100	100	100	100	
ComptrollerStorageR1.sol	100	100	100	100	
contracts/Comptroller/legacy/Diamond/	0	0	0	0	
DiamondConsolidatedR1.sol	100	100	100	100	
DiamondR1.sol	0	0	0	0	... 253,254,256
contracts/Comptroller/legacy/Diamond/facets/	0	0	0	0	
FacetBaseR1.sol	0	0	0	0	... 217,219,227
MarketFacetR1.sol	0	0	0	0	... 329,331,333
PolicyFacetR1.sol	0	0	0	0	... 512,515,516
RewardFacetR1.sol	0	0	0	0	... 236,237,248
SetterFacetR1.sol	0	0	0	0	... 779,780,781
XVSRewardsHelperR1.sol	0	0	0	0	... 122,123,124
contracts/Comptroller/legacy/Diamond/interfaces/	100	100	100	100	
IDiamondCutR1.sol	100	100	100	100	
IMarketFacetR1.sol	100	100	100	100	
IPolicyFacetR1.sol	100	100	100	100	
IRewardFacetR1.sol	100	100	100	100	

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
ISetterFacetR1.sol	100	100	100	100	
contracts/DelegateBorrowers/	100	89.47	100	100	
MoveDebtDelegate.sol	100	91.67	100	100	
SwapDebtDelegate.sol	100	85.71	100	100	
contracts/FlashLoan/interfaces/	100	100	100	100	
IFlashLoanReceiver.sol	100	100	100	100	
contracts/Governance/	73.15	44.87	68.18	68	
TokenRedeemer.sol	97.53	70	100	91.4	... 169,176,180
VTreasury.sol	0	0	0	0	... 65,67,70,72
VTreasuryV8.sol	0	0	0	0	... 90,91,98,99
contracts/InterestRateModels/	72.55	59.09	64.29	74.03	
InterestRateModel.sol	100	100	100	100	
InterestRateModelV8.sol	100	100	100	100	
JumpRateModel.sol	71.43	100	75	78.95	114,115,116,117
TwoKinksInterestRateModel.sol	100	56.25	100	93.33	100,104,173
WhitePaperInterestRateModel.sol	0	0	0	0	... 88,89,90,91
contracts/Lens/	41.12	33.87	36.67	40.53	
ComptrollerLens.sol	88.37	68.18	100	91.07	... 194,238,252
SnapshotLens.sol	0	0	0	0	... 127,129,151
VenusLens.sol	31.85	17.65	25	32.37	... 701,711,733
contracts/Liquidator/	84.05	59.52	86.05	83.25	
BUSDLiquidator.sol	97.56	58.33	91.67	98.04	99
Liquidator.sol	79.51	59.72	83.87	78.48	... 514,515,516
LiquidatorStorage.sol	100	100	100	100	
contracts/Oracle/	100	100	100	100	
PriceOracle.sol	100	100	100	100	

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts/PegStability/	87.91	84.48	85	88.1	
IVAI.sol	100	100	100	100	
PegStability.sol	87.91	84.48	85	88.1	... 424,425,428
contracts/Swap/	92.68	57.14	96.49	87.38	
IRouterHelper.sol	100	100	100	100	
RouterHelper.sol	98.75	70.83	100	92.52	... 308,312,323
SwapRouter.sol	89.76	54.13	95.35	84.65	... 942,943,944
contracts/Swap/interfaces/	100	100	100	100	
CustomErrors.sol	100	100	100	100	
IPancakePair.sol	100	100	100	100	
IPancakeSwapV2Factory.sol	100	100	100	100	
IPancakeSwapV2Router.sol	100	100	100	100	
IVBNB.sol	100	100	100	100	
IVtoken.sol	100	100	100	100	
IWBNB.sol	100	100	100	100	
InterfaceComptroller.sol	100	100	100	100	
contracts/Swap/lib/	100	52.63	100	81.03	
PancakeLibrary.sol	100	50	100	82.61	... 121,141,167
TransferHelper.sol	100	62.5	100	75	18,33,62
contracts/Tokens/	100	100	100	100	
EIP20Interface.sol	100	100	100	100	
EIP20NonStandardInterface.sol	100	100	100	100	
contracts/Tokens/Prime/	95.61	71.59	95.65	96.38	
IPrime.sol	100	100	100	100	
IPrimeV5.sol	100	100	100	100	
Prime.sol	94.87	68.94	95.83	96.35	... 4,1024,1133

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
PrimeLiquidityProvider.sol	97.65	79.55	95.24	96.49	123,215,309,351
PrimeLiquidityProviderStorage.sol	100	100	100	100	
PrimeStorage.sol	100	100	100	100	
contracts/Tokens/Prime/Interfaces/	100	100	100	100	
IPoolRegistry.sol	100	100	100	100	
IPrime.sol	100	100	100	100	
IPrimeLiquidityProvider.sol	100	100	100	100	
IVToken.sol	100	100	100	100	
IXVSVault.sol	100	100	100	100	
InterfaceComptroller.sol	100	100	100	100	
contracts/Tokens/Prime/lib	90.38	75.76	100	89.86	s/
FixedMath.sol	100	50	100	100	
FixedMath0x.sol	88.24	76	100	88.52	... 211,217,223
Scores.sol	90	90	100	100	
contracts/Tokens/VAI/	78.78	52.7	85.96	81.5	
IVAI.sol	100	100	100	100	
VAI.sol	57.69	30	66.67	65.85	... 156,157,158
VAIController.sol	85.02	59.2	97.14	87.21	... 580,581,583
VAIControllerInterface.sol	100	100	100	100	
VAIControllerStorage.sol	100	100	100	100	
VAIUnitroller.sol	44	25	66.67	48.48	... 123,124,126
lib.sol	100	100	100	100	
contracts/Tokens/VRT/	21.32	8.87	25.64	19.66	
VRT.sol	36.71	18.97	52.63	38.04	... 305,306,309
VRTConverter.sol	0	0	0	0	... 153,158,159
VRTConverterProxy.sol	0	0	0	0	... 168,178,180

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
VRTConverterStorage.sol	100	100	100	100	
contracts/Tokens/VTokens/	65.01	51.02	57.89	66.72	
VBNB.sol	0	0	0	0	... 180,181,183
VBep20.sol	75.68	40	70.59	78.05	... 128,129,180
VBep20Delegate.sol	50	25	66.67	42.86	29,40,41,44
VBep20Delegator.sol	26.92	50	26.32	32.14	... 460,498,501
VBep20Immutable.sol	100	100	100	100	
VToken.sol	75.63	54.2	83.87	76.17	... 2,1873,1878
VTokenInterfaces.sol	100	100	100	100	
contracts/Tokens/VTokens/ legacy/	0	0	0	0	
ComptrollerInterface.sol	100	100	100	100	
IProtocolShareReserveV5.sol	100	100	100	100	
VBep20DelegateR1.sol	0	0	0	0	... 30,38,39,42
VBep20DelegatorR1.sol	0	0	0	0	... 522,523,528
VBep20R1.sol	0	0	0	0	... 259,275,284
VTokenInterfaceR1.sol	100	100	100	100	
VTokenR1.sol	0	0	0	0	... 3,1687,1691
VTokenStorageR1.sol	100	100	100	100	
contracts/Tokens/VTokens/ legacy/Utils/	0	0	0	0	
CarefulMath.sol	0	0	0	0	... 76,78,79,82
ErrorReporter.sol	0	100	0	0	... 272,279,281
Exponential.sol	0	0	0	0	... 168,170,179
ExponentialNoError.sol	0	0	0	0	... 188,189,193
contracts/Tokens/XVS/	19.08	8.46	23.26	18.52	
IXVS.sol	100	100	100	100	
IXVSVesting.sol	100	100	100	100	

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
XVS.sol	36.71	18.97	52.63	38.04	... 305,306,309
XVSVesting.sol	0	0	0	0	... 218,223,224
XVSVestingProxy.sol	0	0	0	0	... 151,161,163
XVSVestingStorage.sol	100	100	100	100	
contracts/Tokens/test/	100	100	100	100	
IERC20NonStandard.sol	100	100	100	100	
contracts/Utils/	52.46	30.71	49.61	52.21	
Address.sol	42.86	0	33.33	50	44,66,70,71
CarefulMath.sol	80	66.67	100	84	35,46,77,88
CheckpointView.sol	100	100	100	100	
Context.sol	0	100	0	0	19,23,24
ECDSA.sol	0	0	0	0	... 6,97,98,101
ErrorReporter.sol	66.67	100	66.67	66.67	354,356,363,365
Exponential.sol	54	40.91	53.85	54	... 169,171,180
ExponentialNoError.sol	76.09	62.5	67.65	76.09	... 173,177,181
IBEP20.sol	100	100	100	100	
Ownable.sol	0	0	0	0	... 63,70,71,72
Owned.sol	0	0	33.33	20	13,14,18,19
ReentrancyGuardTransient.sol	100	50	100	87.5	51
SafeBEP20.sol	53.85	33.33	50	53.85	... 41,42,46,50
SafeCast.sol	8.33	4.17	8.33	8.33	... 193,204,205
SafeMath.sol	85	58.33	77.78	85	145,160,161
Tokenlock.sol	33.33	16.67	33.33	33.33	18,20,24,26
TransientSlot.sol	20	100	20	20	... 161,170,179
contracts/VAIVault/	45.95	45.16	51.85	50.49	
VAIVault.sol	75.56	51.85	73.68	78.79	... 217,218,236
VAIVaultErrorReporter.sol	0	100	0	0	26,28,35,37

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
VAIVaultProxy.sol	0	0	0	0	... 125,135,137
VAIVaultStorage.sol	100	100	100	100	
contracts/VRTVault/	47.66	36.29	53.33	48.59	
VRTVault.sol	62.2	40.18	72.73	64.49	... 277,304,305
VRTVaultProxy.sol	0	0	0	0	... 144,154,156
VRTVaultStorage.sol	100	100	100	100	
contracts/XVSVault/	60.67	50	55.71	63.37	
XVSSStore.sol	60	46.15	66.67	60.71	... 1,93,94,125
XVSVault.sol	67.73	53.13	62.26	71.2	... 868,921,922
XVSVaultErrorReporter.sol	0	100	0	0	26,28,35,37
XVSVaultProxy.sol	0	0	0	0	... 125,135,137
XVSVaultStorage.sol	100	100	100	100	
contracts/external/	100	100	100	100	
IProtocolShareReserve.sol	100	100	100	100	
IWBNB.sol	100	100	100	100	
contracts/lib/	100	71.43	100	88.89	
Currency.sol	100	90	100	92.86	58
approveOrRevert.sol	100	25	100	75	25
All files	54.37	42.09	52.75	55.95	

Changelog

- 2026-03-20 - Initial report
- 2026-03-20 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp’s mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp’s team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked

with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PandcakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

